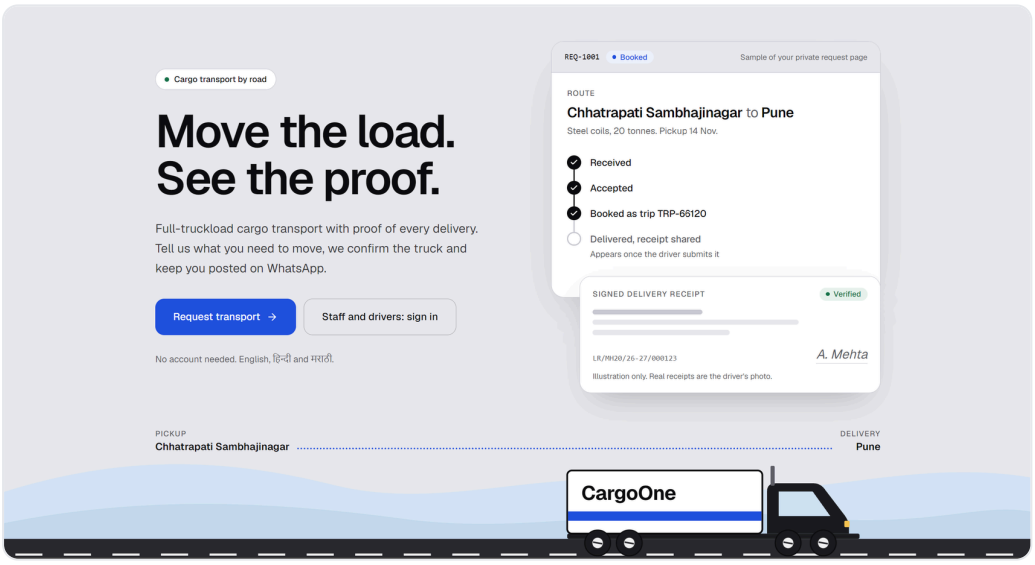
CargoOne

Technical Design Document
Proof-of-delivery capture, AI verification and Tally billing for Company A

GitHub	**https://github.com/priyadarshi7/CargoOne**
Live app	Not deployed. It runs locally and in CI; `deployment.md` has the steps.
Author	Priyadarshi (solo build)
Date	9 November 2026
Brief	Company A, Chhatrapati Sambhajinagar: POD capture, AI verification and Tally billing (Software Engineering core build)



1. Summary
2. The client and the problem
3. Architecture and why
4. Roles and access
5. Data model
6. The shipment flow
7. Works offline
8. The AI part
9. AI benchmark
10. Integrations
11. GPS, tracking and the driver's side
12. UI and design decisions
13. Security and privacy (DPDP Act 2023)
14. Testing, and the sample data

Appendix A. Feature checklist

1. Summary

CargoOne turns the proof of delivery (POD) from a paper that arrives 10 to 20 days late into a photo that is checked, billed and visible on the day of delivery. The driver photographs the signed lorry receipt (LR) at the consignee's gate with one big button. The phone compresses the photo, stamps it with time and GPS, and sends it, **even with no network**. An AI reads it while the driver is still there. Fixed rules, not the AI, decide: verified, retake, or a person looks. Verified trips go to a billing queue, become an invoice with every trip's POD photo attached, and are pushed into Tally with one click.

The core slice from the brief works end to end. Around it I built the whole shipment, from a client's request to the paid invoice, because a POD only matters inside the trip that produced it. Items the brief marks out of scope (live GPS, trip planning, driver expenses) exist as light, clearly labelled extras.

The brief asks for	Status
Driver app, one big button, photo stamped with GPS and time	Built. Installable PWA, icons, Hindi, Marathi and English voice prompts
Offline capture, images under 300 KB	Built. Phone queue, in-order retry, never counted twice (section 7)
SMS upload link for hired drivers	Built. Private one-trip link; Twilio sends it, the link is also shown on screen
AI POD check with retake	Built. Groq main reader, local fallback, fixed rules, retakes capped at 2
Review queue, billing queue, Tally export, POD images on invoices	Built. One-click Tally push; invoice PDF carries every POD photo
Dashboard of missing PODs and POD-to-invoice days by branch; client link	Built. Client link and client portal too
Daily trip import with LR and e-way bill	Built. Preview first, write only on confirm
DPDP Act 2023	Built. Notice, request register, erasure, 8-year retention

> Every number from the brief is used as given. Where I had to assume something, the text says so.

2. The client and the problem

Company A, Chhatrapati Sambhajinagar: full-truckload transporter for cement, steel and packaged goods. 140 own trucks plus about 200 hired, 210 staff, revenue Rs 75 to 90 crore a year. Drivers bring the signed LR back 10 to 20 days late, WhatsApp photos get lost, and Tally billing waits because clients reject invoices without the POD.

Item	Working (the brief's numbers)	Result
Annual freight	2,200 trips a month x Rs 32,000 x 12	Rs 84.48 crore
Working capital freed	84.48 crore x 14 days / 365 (18 days down to 4)	Rs 3.24 crore
Interest saved	3.24 crore x 12%	Rs 38.9 lakh a year
Lost or disputed PODs	1.5% = 396 trips x Rs 16,000 (half the freight)	Rs 63.36 lakh a year
Total		about Rs 1.02 crore a year

Targets: 80% of PODs in the system within 24 hours; POD-to-invoice from 18 days to 4; lost or disputed PODs from 1.5% to 0.5%. The dashboard measures exactly these three. Targets are settings (`TARGET_POD_WITHIN_24H_PCT`, `TARGET_POD_TO_INVOICE_DAYS`, `TARGET_LOST_PCT`), not hard-coded.

User	Needs	CargoOne gives
Driver (basic phone, low literacy)	Take one photo and be done	One big button, icons, spoken prompts, no typing, offline
Hired driver	The same, no install	Private SMS link for one trip
Billing executive	Know what can be billed; handle doubtful PODs	Review queue, billing queue, invoices, Tally
Operations head	Book trips, chase missing PODs	Planning board, requests, Needs attention, live map
Managing Director	See whether targets are met	Performance dashboard, working capital, digest
Client and receiver	Know where the load is; see proof; pay	Portal, shipment page, delivery code, online payment

3. Architecture and why

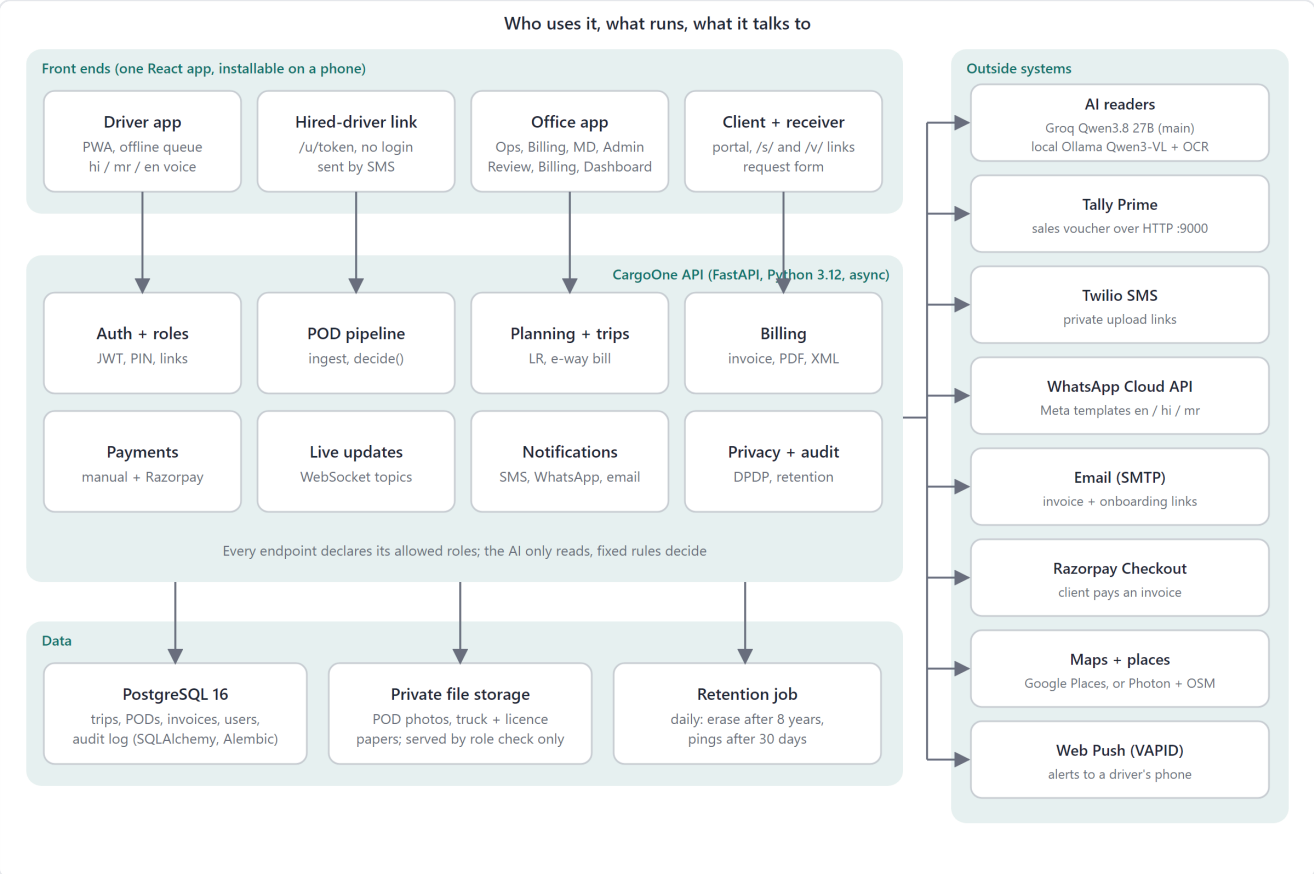


Figure 1. Front ends, the API, data, and the outside systems.

One React web app (installable on a phone) talks to one FastAPI service over HTTPS and one WebSocket. The service owns a PostgreSQL database and a private file store. Every outside system sits behind an adapter with a live mode and a simulated mode, switched by one setting.

Part	Choice	Why

Front end	Vite, React 19, TypeScript, Tailwind v4	One codebase for phone, desk and client browser
Phone app	PWA, service worker, IndexedDB	No app store; installs from a link; works offline
Back end	FastAPI, Python 3.12, async SQLAlchemy 2	Typed validation by default; async so AI and Tally calls do not block
Database	PostgreSQL 16, Alembic (27 revisions)	Money and audit need constraints (the retake cap is also a DB constraint)
AI	Groq Qwen3.8 27B; local Ollama Qwen3-VL 4B + glm-ocr / RapidOCR; Gemini; mock	Chosen by measurement (section 9); local keeps it working with no network or data leaving
Live updates	In-process WebSocket hub	Only topic names travel on the socket; each screen refetches with its own login
Maps	Leaflet + OpenStreetMap, Photon search, Google Places optional	Free by default
Quality	pytest, vitest, Playwright, GitHub Actions	478 API tests, 49 web unit tests, 30 browser spec files

- **The AI reads; rules decide.** `api/app/pod/decision.py` is plain code. Swapping the model cannot loosen the rules.
- **The photo is the source of truth.** Stored bytes are never changed; a person's correction is kept beside the AI's reading.
- **Invoices are never sent automatically.** Create, push to Tally and send are three human clicks.
- **Roles are enforced in the API,** not only hidden in screens.
- **One process, one database.** Right-sized for this client. Known limit: the live-update hub is in memory, so it runs as one process.

4. Roles and access

Area	Driver	Billing	Ops	MD	Client
Capture POD, loading photo, expenses, own messages	Yes				
Review queue: approve, retake, reject		Yes	Yes	View	
Reopen a rejected POD; cancel an unsent invoice			Yes		
Billing queue, invoices, payments, Tally		Yes	Yes		Own invoices
Undo a payment; staff accounts (admin only)					
Book and replan trips, assign truck and driver		View	Yes		
Requests and quotes		View	Yes		Own
Setup: trucks, drivers, vendors, clients, branches		View	Yes		
Performance dashboard			Yes	Yes	
Live map and driver location			Yes	Yes	Own, rounded
Trip import		Yes	Yes		
AI benchmark		Yes	Yes	Yes	

Privacy requests (the officer; admin too)	Yes

No login needed: a hired driver (private link, one trip, 72 hours), a receiver (shipment link with their delivery code, never a price or invoice), and a requester (signed status link). Drivers get 404, not 403, for trips that are not theirs. PINs and passwords are Argon2 hashes; five wrong sign-ins lock for 15 minutes; link tokens are 256-bit and only their SHA-256 is stored.

5. Data model

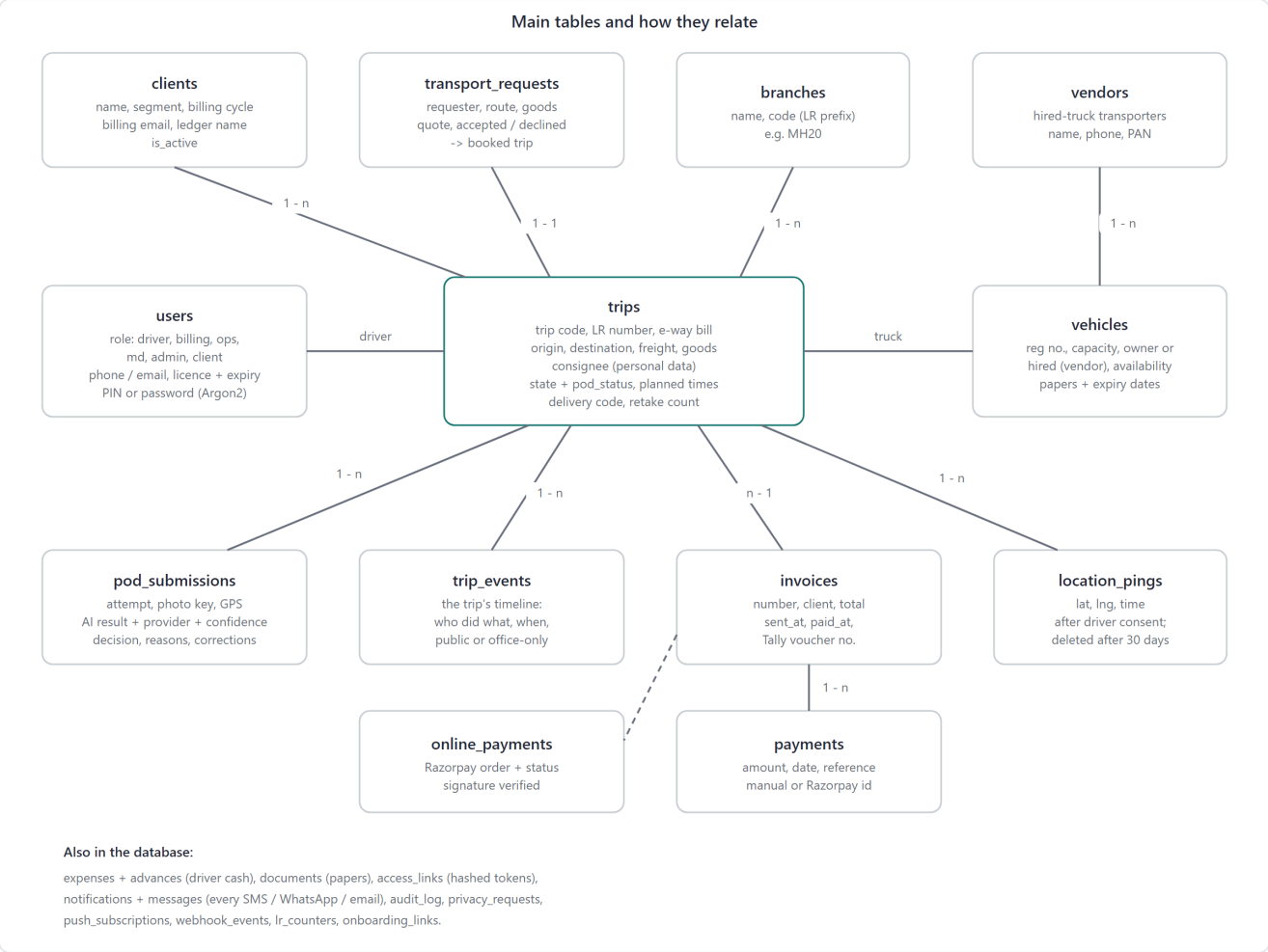


Figure 2. The main tables. The trip is the centre.

Table	Holds
trips	Trip code, LR number, e-way bill (one live trip only), route, freight, goods, consignee (personal data), state, POD status, planned times, delivery code
pod_submissions	One row per photo: storage key, GPS, AI reading, provider, confidence, decision, reasons, corrections. Retries are idempotent
trip_events, audit_log	The trip timeline (public or office-only); who did what
invoices, payments, online_payments	Invoices, manual and Razorpay payments, part payments allowed
users, vehicles, vendors, clients, branches	People with roles and licences; own and hired trucks with papers and expiry; hired-truck vendors
transport_requests	Incoming requests with quote state; becomes a trip

location_pings	Position after consent; deleted after 30 days
expenses, advances, documents, access_links	Driver cash with photos; truck and licence papers; hashed link tokens
notifications, messages, privacy_requests, lr_counters	Every SMS, WhatsApp and email; DPDP register; LR serials

LR numbers are `LR/<branch code>/<financial year>/<serial>` (for example `LR/MH20/26-27/000123`), restart each April, and are issued by a single insert-or-increment so two people can never get the same one.

6. The shipment flow

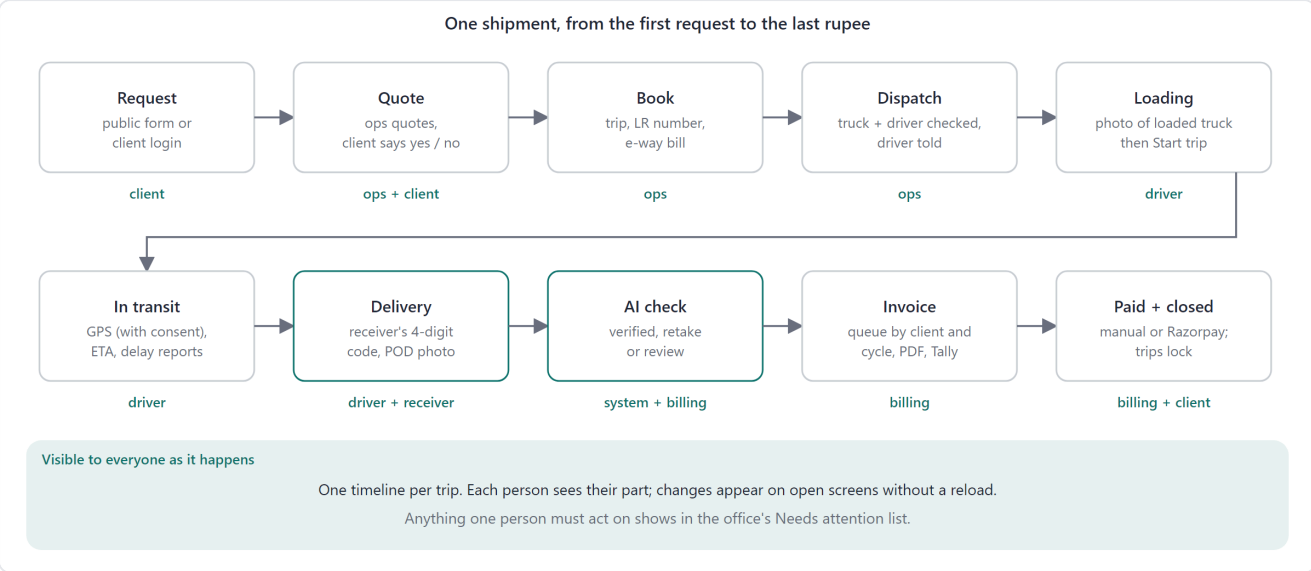


Figure 3. One shipment, from request to paid invoice.

Step	What happens
Request	A new company uses the public `/request` form (consent stored with a time); an existing client asks from `/portal`. Ops accepts with an optional quote; the requester answers yes or no on their status link; no trip can be booked from a quote before a yes.
Book	LR number issued; e-way bill and validity entered (must cover the trip, one live trip only); consignee name, phone and email required. Truck and driver checked for papers, capacity and clashes. Driver told by app, WhatsApp and push.
Loading	Driver photographs the loaded truck and LR; only then does **Start trip** work, from 3 hours before the planned start.
On the road	Position shared with consent; shipment page shows progress, timeline, position rounded to ~100 m and an arrival estimate. Delay or breakdown in a few taps (no typing).
Delivery	Driver taps **I have reached**; the receiver reads a 4-digit delivery code from their link; the driver enters it (five wrong tries lock 15 minutes; only the office can bypass, with a reason). Then the POD photo.
Hired driver	Office presses **Send link**; a one-trip, 72-hour SMS link opens the same capture engine with no login or install.
Live updates	Open screens refresh without reload; only a topic name is published and each screen refetches under its own login.

Needs attention	One list for everything waiting on someone: new request, a client's no, trip leaving with no driver, driver has not seen the job, breakdown, late trip, shortage at the door, POD missing or waiting, invoice not sent for a day, unpaid over 30 days, failed message.

7. Works offline

Highways have dead zones, so the driver app is built **offline first**. Nothing the driver does at the gate depends on a network.

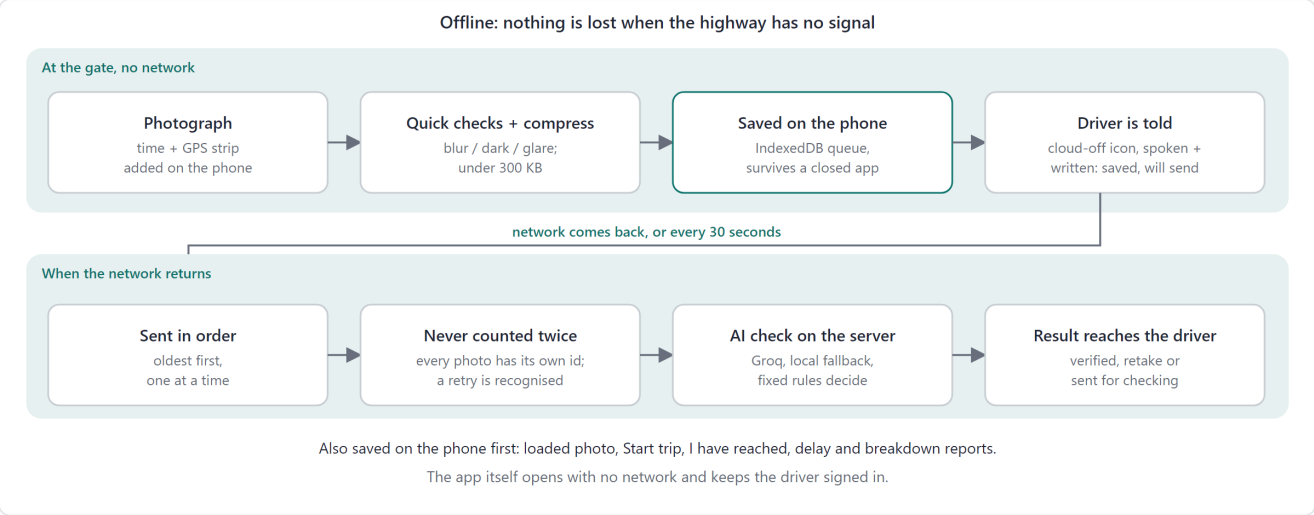
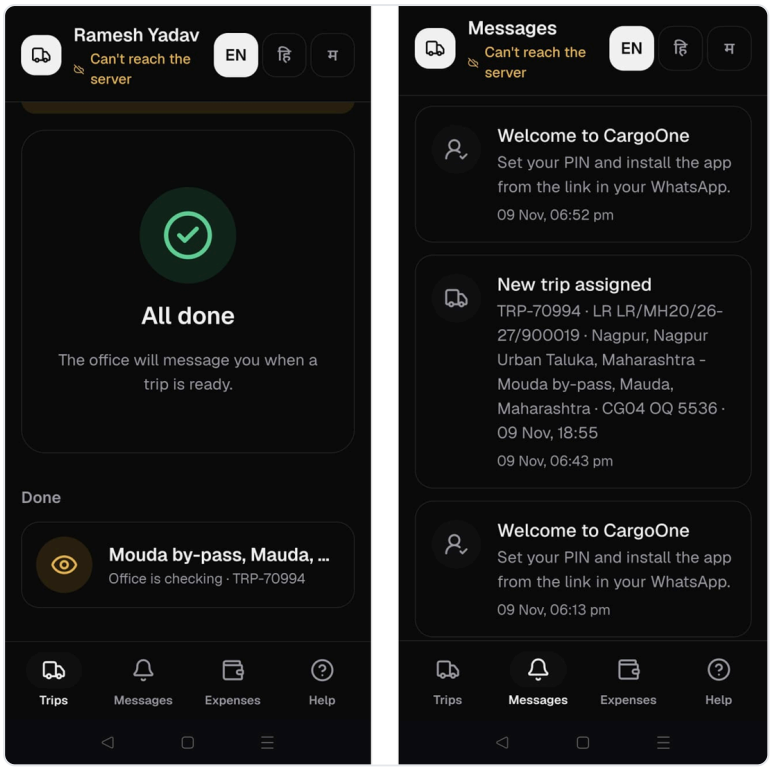


Figure 4. Offline capture: saved on the phone, sent when the network returns.

Part	How it works
The app opens offline	A service worker caches the app shell (screens, scripts, icons). The driver stays signed in. Data is never cached, so it is always live when online.
Capture offline	Photo, quick checks (blur, dark, glare, blank), compression under 300 KB and the time and GPS strip all happen on the phone. If GPS is slow or denied (4 s) capture never blocks.
The queue	Saved in IndexedDB, survives a closed app, kept in order, retried when the network returns and every 30 seconds. A permanent error on one item is isolated and does not block the rest.
Never twice	Every photo carries its own id, so a retry is recognised and one POD is never counted two times.
Other actions	Loaded photo, Start trip, I have reached, delay and breakdown reports are saved on the phone first, the same way.
Driver feedback	A cloud-off icon with a spoken and written message says saved on the phone and will send. A counter shows what is waiting.
Hired-driver link	The SMS capture page uses the same engine and queue.
Time	Online, the stamp uses the server's clock (the app learns the phone's offset). Offline it uses the phone clock and prints "(phone clock)".
What waits for a network	The AI check and its result; they happen when the queued photo uploads. The office screens need a connection.

Still open: exponential back-off and the Background Sync API, a storage-quota warning, and an in-app update prompt. The queue and compression are tested in a phone-sized browser, including an offline Playwright test.



Screenshot: the driver app with no connection ("Can't reach the server"). It still opens, shows the trip as Office is checking, and keeps the messages. Left: trips. Right: messages.

8. The AI part

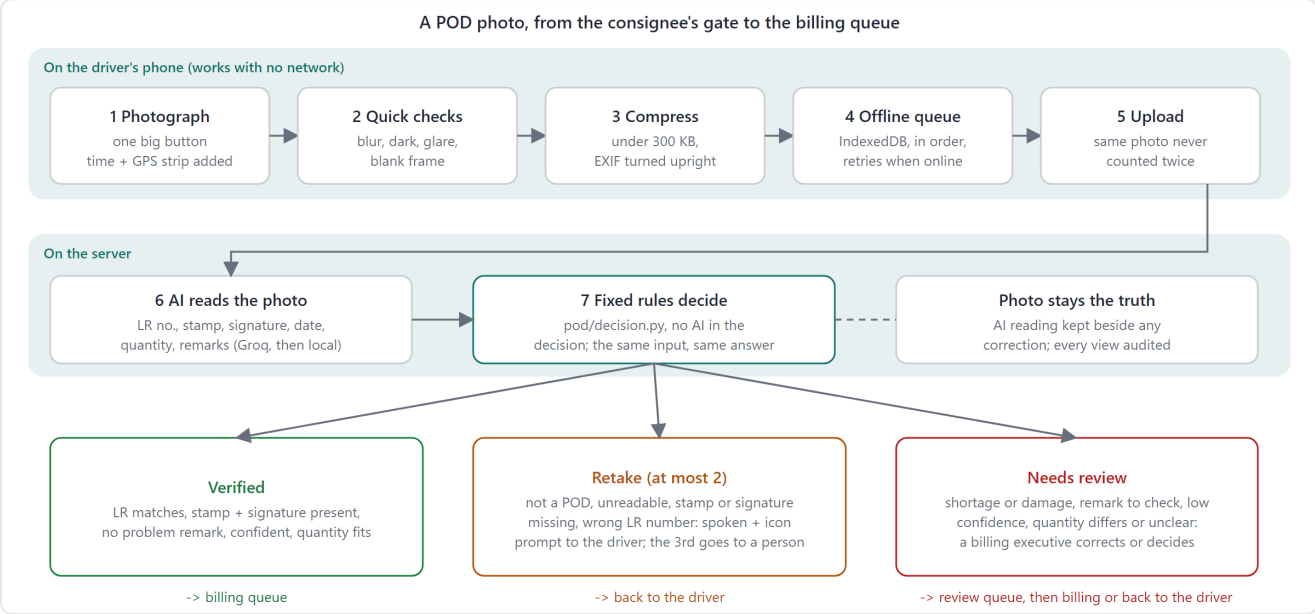


Figure 5. A POD photo from the gate to the billing queue.

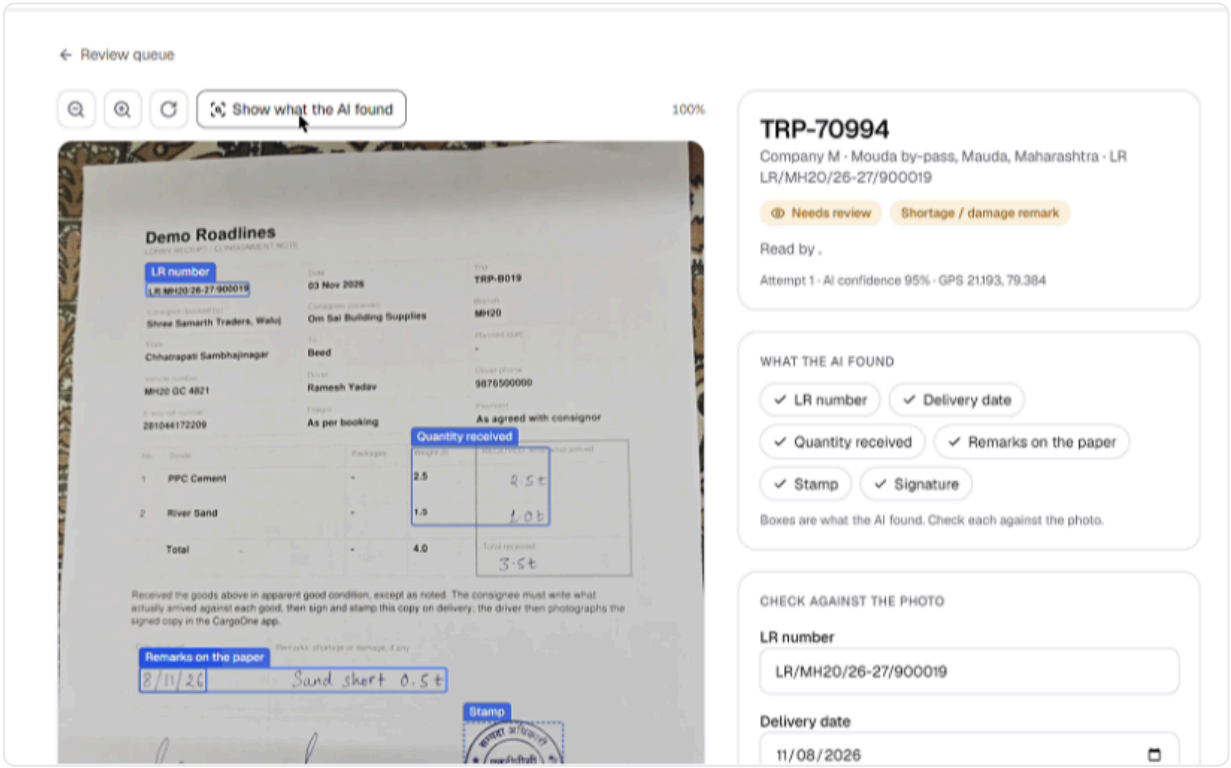
What the AI reads: is it a delivery paper, is it readable, the LR number, consignee stamp, signature, delivery date, quantity received (line by line for several goods) and written total, handwritten remarks, a shortage or damage flag, and its confidence. The prompt never tells the model which LR number to expect.

Outcome	When	What happens

Retake	Not a POD; unreadable; stamp or signature missing; LR differs from the trip	Spoken and written prompt with an icon. Capped at 2 (server and DB constraint); then a person decides
Needs review	Shortage or damage remark; any remark not plainly fine; low confidence; quantity differs from the booking (5% for weights, exact for counts); several goods or units that cannot be reduced to one figure	A billing executive sees the photo beside what the AI read, can correct fields, approve, retake or reject
Verified	Everything clear	Into the billing queue; still openable later

If the AI is wrong or unsure	Defence
Misreads a shortage remark as harmless	Any remark not plainly fine goes to a person. Caught S13, where Groq read a Marathi shortage remark as other words
Completes or invents the LR number	Expected number is never shown to the model; the local reader accepts it only when OCR sees it exactly
Confident but wrong quantity	Read two ways (lines and quoted text); any disagreement with the booking counts
Cloud model down or rate-limited	Falls back to the local reader; if both fail the POD goes to a person
Says verified when it should not	Counted as an unsafe error; target zero
Driver keeps retaking	Third goes to review

Labels on the photo. The review screen draws what the AI found on the photo: boxes for LR, date, quantity and remarks (local text reader) and for stamp and signature (local vision model), with a chip per finding that highlights on hover. The local reader draws them as it reads; for Groq, which returns only values, the app places them once on first open and keeps them. They aid the reviewer and never change the decision.



Screenshot: the review screen. What the AI found is drawn on the photo (LR number, quantity, remarks, stamp) with a chip per finding; the fields on the right can be corrected.

Readers: `groq` (main), `local` (Ollama, no network), `gemini`, `mock` (default and tests). Outside development a cloud reader runs only when `CLOUD_AI_CONFIRMED=true`, because a photo may show names and signatures.

9. AI benchmark

I chose the model by measurement. The measure that matters to the client is never letting a bad POD through.

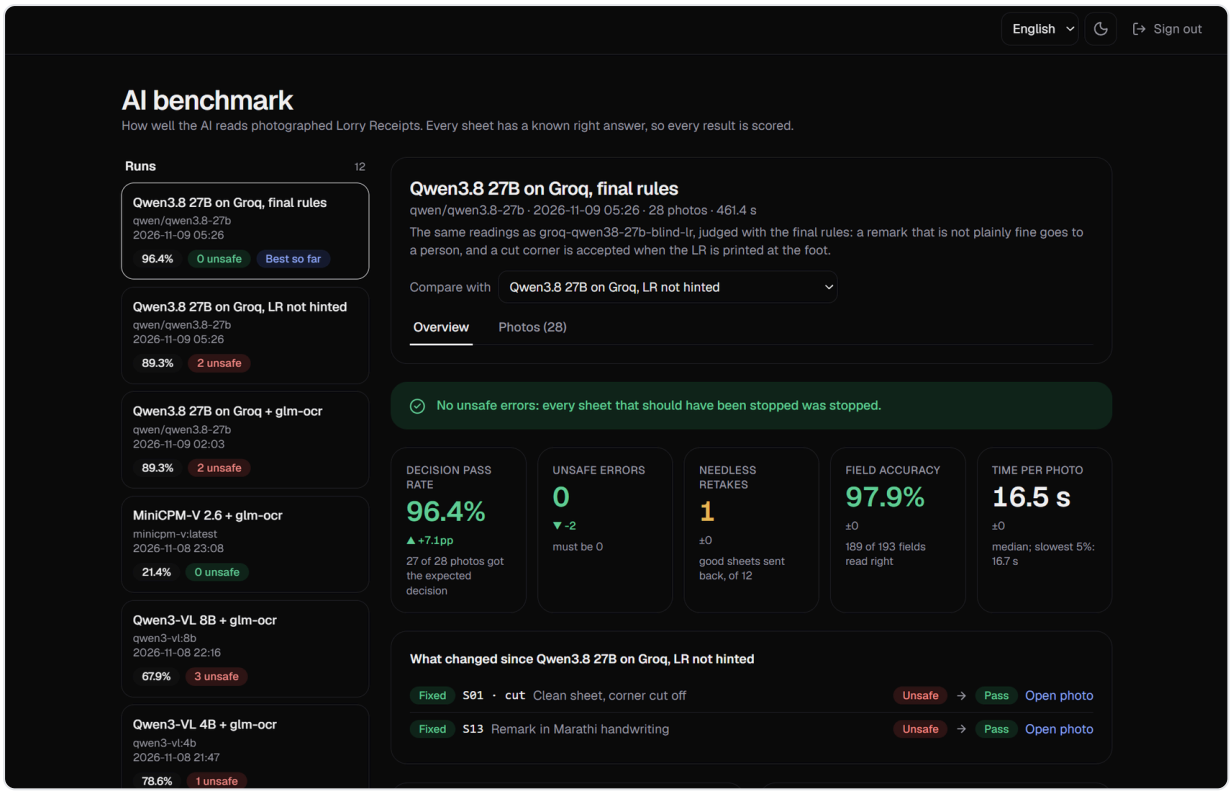
How the dataset was made

1. **22 sample LRs** (`docs/benchmark/lr_forms.pdf`): the app's own printed LR with fake data and serials from 900001, so a test sheet cannot be mistaken for a real one.
2. **Receiver part left empty and filled by hand** in different inks and handwriting, one case per sheet: clean, no stamp, no signature, empty, wrong LR, shortage with remark, damage, short count with no remark, weighed loads, blank date, Hindi and Marathi remarks, faint or messy stamp, a harmless remark, and six multi-goods cases.
3. **Photo conditions:** sheet S01 photographed seven ways (flat, angled, dim, glare, crumpled, blurred, corner cut off). 21 other sheets plus 7 shots makes **28 photos**.
4. **Answer key** (`answers.csv`): expected stamp, signature, date, quantity, items, total, remark and decision per sheet, computed by the app's own rules. Judgement-call sheets (S14, S15, S22) accept more than one answer.
5. **Harness and screen:** `app.devtools.benchmark` saves resumable runs; the **AI benchmark** tab shows every run with expected against actual per photo.

Counted: *unsafe* (a sheet that should not verify would have), *needless retake* (a good sheet sent back), *fields right*. Unsafe must be 0.

Run	Pass	Unsafe	Needless	Fields	Time
Qwen3.8 27B on Groq, final rules (chosen)	**96.4% (27/28)**	**0**	1	97.9%	16 s
Qwen3.8 27B on Groq, before final rules	89.3%	2	1	97.9%	16 s
Qwen3-VL 4B + glm-ocr (local)	78.6%	1	1	84.5%	34 s
Qwen3-VL 4B + glm-ocr, before schema fix	85.7%	3	1	85.0%	32 s
Qwen3-VL 8B + glm-ocr (local)	67.9%	3	3	81.3%	57 s
Qwen3-VL 8B + glm-ocr, before schema fix	71.4%	8	0	53.9%	53 s
Qwen3-VL 4B + RapidOCR (first local reader)	71.4%	3	3	76.2%	12 s
MiniCPM-V 2.6 + glm-ocr (local)	21.4%	0	12	57.5%	50 s

- **glm-ocr was the biggest win** for the local reader: it reads handwriting; RapidOCR reads only print.
- **A bigger local model was not better** (8B scored below 4B and does not fit a 6 GB card).
- **The benchmark found a real bug:** the local answer format required six fields, so larger models skipped quantity, date and remarks. All fields are now required.
- **Two readers together** (verify only when both agree) would score 26 of 28 with 0 unsafe; it costs a cloud call plus the local model per POD, so it is left as an owner decision.
- **Honest limits:** 27 of 28 is tuned on this paper (two rules were adjusted after the first Groq runs), on printed sheets with my handwriting. Treat it as tuned, not a forecast: real PODs will be harder. Gemini ran out of free quota; Llama 3.2 Vision could not load; Groq's free tier is about three photos a minute.



Screenshot: the AI benchmark tab (dark theme). Every run is listed; the chosen run shows 96.4%, 0 unsafe errors, 97.9% of fields right, and what changed since the previous run.

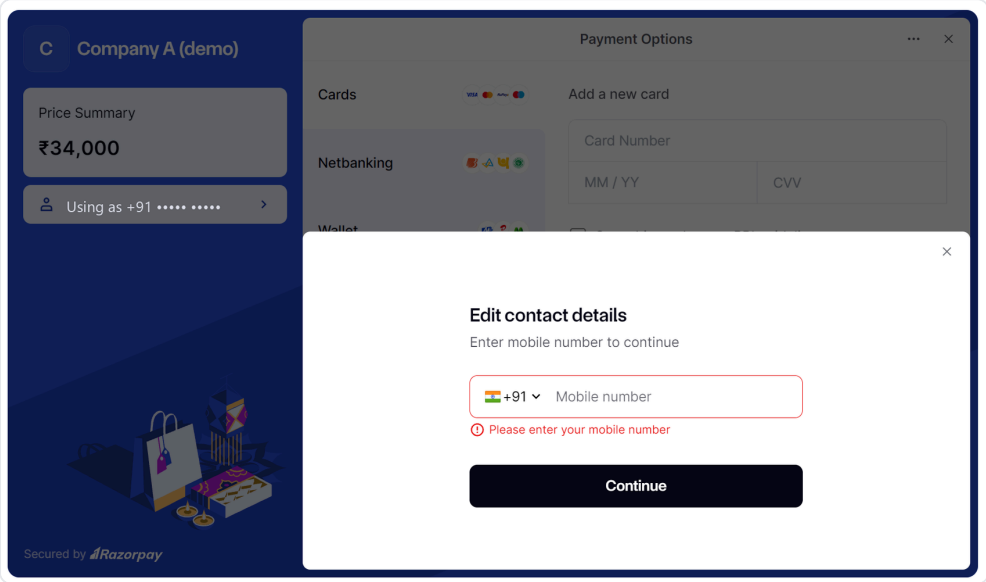
10. Integrations

Each has a **live** mode (credentials set) and a **simulated** mode (default): nothing leaves the machine, the screen says so, and the app never pretends to send.

Integration	What it does	State here	Settings
Twilio SMS	Hired driver's private link; reminders for a POD still owed	Code complete, hard ceiling on real texts. **Simulated:** India needs a registered sender (DLT). Link always shown on screen	`SMS_PROVIDER`, `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_FROM`, `SMS_MAX_TOTAL`
WhatsApp (Meta Cloud API)	Driver link, new trip, change, request received and updated, shipment updates to client and receiver, in their own language	Code complete, 6 notices x 3 languages = 18 templates. **Recording mode** until Meta approves: logged as Not sent, with Open WhatsApp	`WHATSAPP_PROVIDER`, `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_TEMPLATE_*`
Email (SMTP)	Invoice link, one-time onboarding and reset links	Code complete; simulated until set	`EMAIL_PROVIDER`, `SMTP_*`
Tally Prime	One-click Sales voucher; XML download	**Working** against a local Tally on port 9000	`TALLY_URL`, `TALLY_COMPANY`, `TALLY_SALES_LEDGER`
Razorpay	A client pays an invoice online (UPI, card, netbanking)	Code complete, unit-tested; **needs test keys** end to end. Not for driver payouts	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`

Groq, Ollama, Gemini	Read the POD photo	Groq needs a key; Ollama is local	`AI_PROVIDER`, `AI_FALLBACKS`, `GROQ_API_KEY`, `OLLAMA_*`, `CLOUD_AI_CONFIRMED`
OpenStreetMap, Photon, Google	Place search and exact pins; live map; driver's Navigate button	**OpenStreetMap and Photon work with no key**; Google optional	`PLACES_PROVIDER`, `GOOGLE_MAPS_*_KEY`
Web Push (VAPID)	Alerts on a driver's phone	Live when both keys are set	`VAPID_*`

- **Twilio:** every real text is counted in the audit log and stops at `SMS_MAX_TOTAL`. If Twilio refuses, the dialog shows why and the link still works. The driver's mobile is not stored (masked in the audit log).
- **Tally:** looks first for a voucher with the same Reference (our invoice number), because Tally does not refuse duplicates; asks before creating the party (Sundry Debtors) or sales ledger; sends one Sales voucher; the date must fall in the company's period (Educational mode accepts only the 1st, 2nd and last day of a month). GST lines are not in the voucher, and a hosted server needs a connector to reach Tally.
- **Razorpay:** the server sets the amount; a payment counts only after the signature verifies and Razorpay confirms it is captured for this order and amount; applied once however often reported; money after the invoice is settled appears in Needs attention to refund. Refunds are made in Razorpay.
- **Maps:** place fields suggest while typing and save exact latitude and longitude; **Pick on map** moves a pin or uses current location; Navigate goes to that exact point. Google results are never cached (its terms); the OpenStreetMap fallback is cached a day.



Screenshot: Razorpay Checkout (test mode) opened from an invoice. The amount is set by the server from the invoice balance; the payer chooses card, netbanking, wallet or UPI. The phone number is masked.

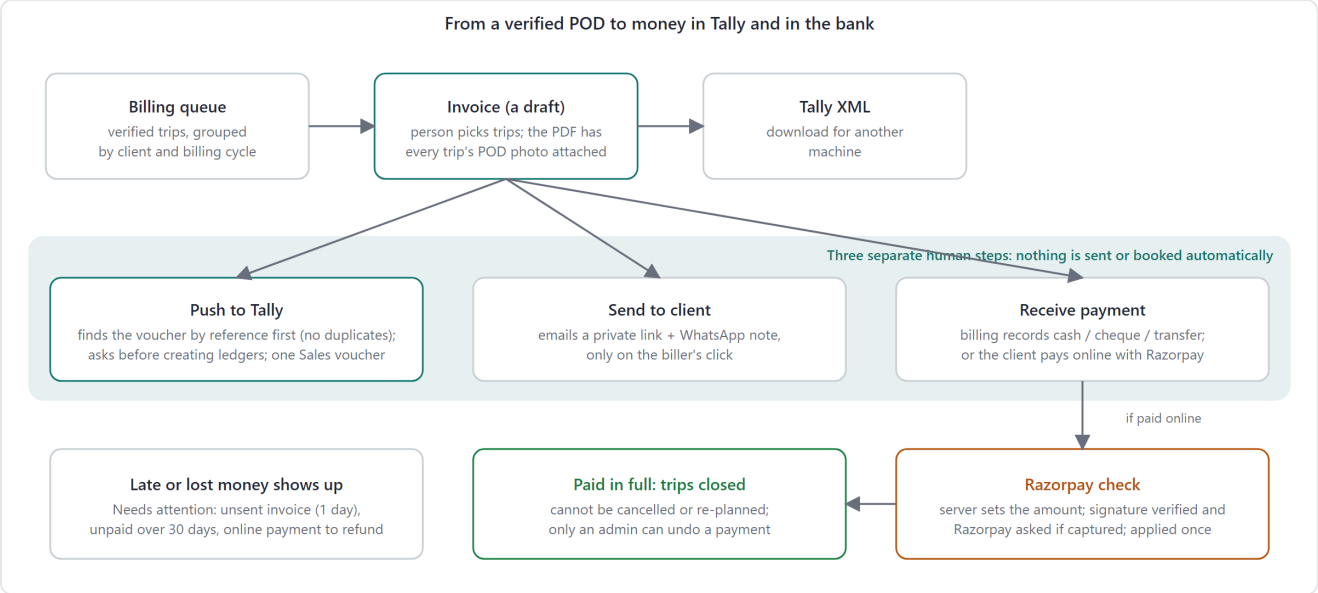
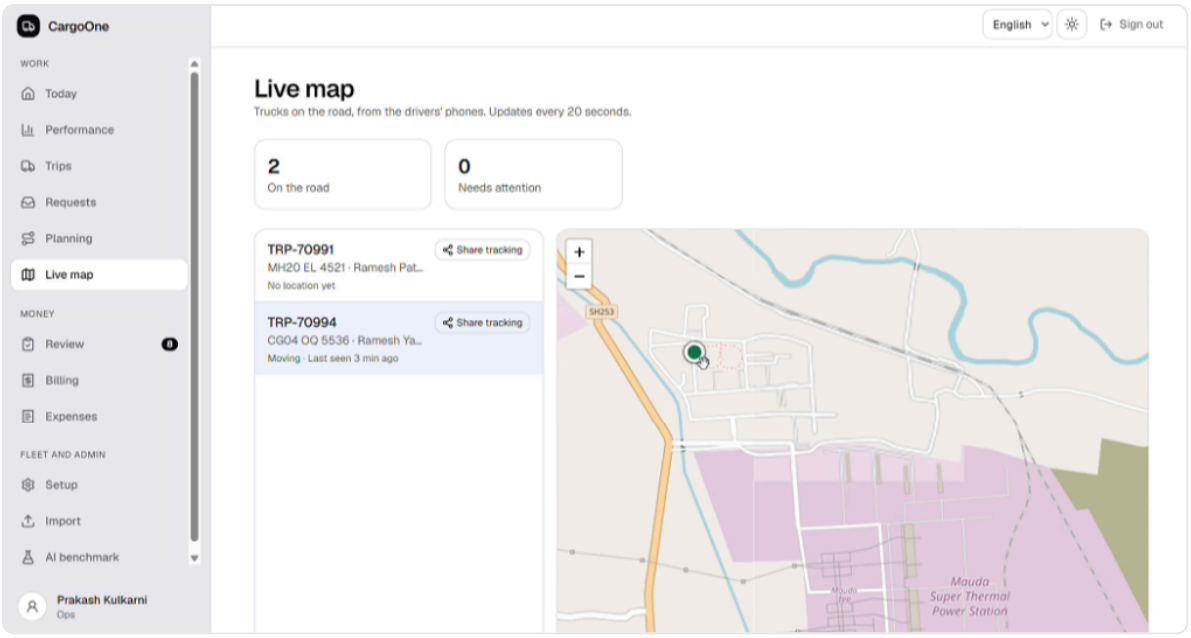


Figure 6. From a verified POD to Tally and the bank.

- **Billing queue** groups verified trips by client and cycle (weekly, fortnightly, monthly). The invoice PDF attaches every trip's POD photo.
- **Send to client** emails a private link and a WhatsApp note on the biller's click; **I sent it myself** records one sent another way. Billing records cash, cheque or transfer (part payments allowed); when payments cover the total the trips close.
- **Ways out of mistakes:** reopen a rejected POD with a reason; cancel an unsent invoice (its trips return to the queue); only an admin can undo a payment.

11. GPS, tracking and the driver's side

Feature	How it works
GPS stamp on every photo	Position and time printed in a strip below the paper; the AI is told to ignore the strip. Loading photos carry it too
Distance check	A delivery photo taken more than `POD_MAX_DISTANCE_KM` (25) from the destination adds a tip for the reviewer and the driver
Consent	Asked as soon as there is a trip; Not now lasts the day. Collected only after yes and only while a trip is on the road
Movement states	Moving, stopped (within 150 m for 90 minutes) or no signal (15 minutes), by pure tested rules
Arrival estimate	Distance x 1.3 road factor / 40 km/h; the timeline says so when it passes the planned end or the e-way bill's validity
Who sees it	Ops and MD on the live map (not billing); every path view audited; the client sees a position rounded to ~100 m
Retention	Fixes deleted after 30 days



Screenshot: the live map. Trucks on the road with their last position and state (moving, stopped, no signal); a Share tracking button makes a link for the client.

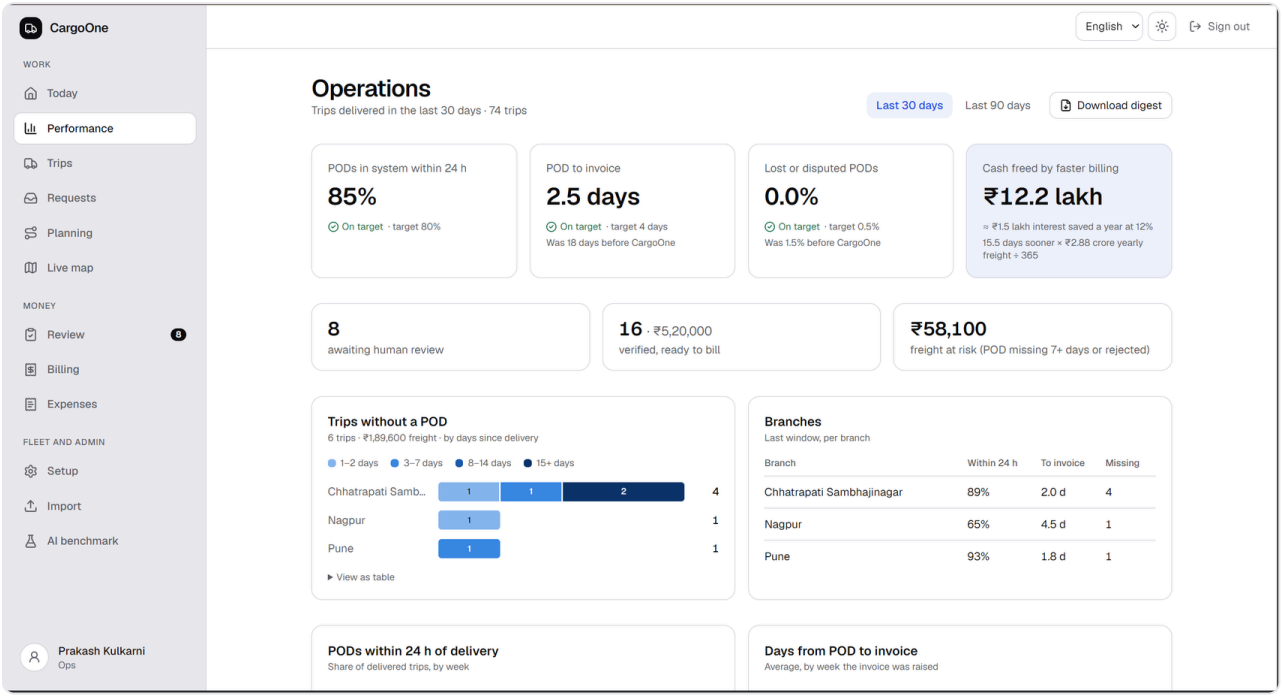
- **Driver home:** one trip, one big button; the truck on the road first, then a delivery waiting for its photo, then the next. Four steps: Loaded, On the road, Reached, Delivered.
- **Voice and icons:** prompts spoken in Hindi, Marathi or English and also written; green tick done, amber arrow retake, eye sent for checking, cloud-off saved on phone; language switchable on the result screen. Wording reviewed by the owner.
- **Sign-in:** phone and PIN; the first PIN is one-time. **Help:** one-tap Call the office (`OFFICE_PHONE`), Help tab, messages from the office with a badge, Web Push alerts.
- **Expenses and advances:** the driver records an expense with a photo; the office approves; advances are recorded by the office; a **Cash with you** card shows advances, approved, waiting and balance. Hidden when there is no money activity; not shown to link drivers.
- **Not connected: driver payouts.** Razorpay is used only for a client paying an invoice; money to drivers moves outside CargoOne.

12. UI and design decisions

Decision	Why
One accent on a neutral base; green, amber, red only for done, retake, problem. Dark theme	A restrained palette makes the status colours carry meaning
Driver screen: one primary action, no forms; every result spoken, written and iconed	Drivers read poorly; never colour alone, never text alone
Review: large photo left, editable fields right, keys A / R / X, labelled boxes with hover chips	The photo is the truth; a biller does this all day
Counts match everywhere	Awaiting review on Today = the Review queue = the dashboard; missing and ready-to-bill use one rule on every screen
Filters with counts on every setup list (for example trucks by paper status: valid, due in 30 days, expired)	A fleet of hundreds must be narrowable

Loading, empty and error states on every screen; phone-first layouts	No blank screens, no dead buttons
English, Hindi and Marathi interface; a test fails the build if any key is missing	The brief names Hindi and Marathi
Live updates instead of reload buttons	Ops should not refresh to see a POD arrive

Key screens: Driver home and result; Today with Needs attention; Review list and detail; Billing and invoices; Performance (three KPI tiles against target, branch league, weekly trend, age buckets, working capital, one-page digest); Planning, Trips, Requests; Setup (trucks, drivers, vendors, clients, branches, staff, messages); Live map; Import; Expenses; AI benchmark; Privacy; Client portal and shipment page; public landing, request form and build-progress page.



Screenshot: the Performance dashboard. The three targets as tiles with On target / Below target, cash freed, counts that match the Today tab, trips without a POD by age and branch, and the branch table.

13. Security and privacy (DPDP Act 2023)

Area	In place
Access	Roles on every endpoint; 404 for others' trips; hashed scoped link tokens; guessing rate-limited
Secrets and input	None in the repo; `.env` git-ignored; non-dev refuses a default or short `JWT_SECRET`; typed validation on every body; the public form has a bot field and per-address and per-phone limits
Images	Private storage, role-checked, `no-store`; type from bytes; size limits; path-traversal guard
Logs	No consignee names or phones in lists, logs or AI errors; message-log numbers masked
Audit	Uploads, photo views, review decisions, invoices, exports, links, every Tally push
Notice and rights	Public `/privacy` in three languages; request form; admin and MD register with lookup, erase and close; consent timestamped; location only after yes
Retention	Daily job erases consignee details, photos and AI readings 8 years after a trip closes (trip, LR, invoice and money stay for tax); declined requests after 180 days; location after 30

Cloud AI gate	A photo reaches Groq or Gemini only after `CLOUD_AI_CONFIRMED=true`
Safety rails	Demo scripts refuse outside dev and test; tests use their own database and force the mock AI

> An engineering record, not legal advice; confirm obligations with counsel. Open owner items are in `docs/16-dpdp.md`.

14. Testing, and the sample data

478 API tests (decision rules, quantity comparison, LR matching, payments, role gates, Tally XML, imports, privacy, links, live updates, dead-end recovery, location rules), **49 web unit tests** (compression, offline queue, i18n coverage), **30 Playwright spec files** in real Chromium including a phone-sized driver project (every role's pages, offline capture, review, billing, requests, map picker, tracking, privacy, live update without reload). GitHub Actions runs lint, migrations, API tests, the web build and the browser tests on every push. Tests are pinned to IST.

Sample data is fabricated and deterministic, made by scripts and never committed. The brief's three trips are seeded exactly. `app.seed` (branches, those trips, logins), `app.seed_fleet` (8 own and 8 hired trucks, 6 vendors, 12 drivers with licences, 5 clients, staff, a few papers deliberately due or expired), `app.demo_data` (about 140 trips over 75 days with rendered POD photos, invoices and payments; no truck or driver on two trips at once), `app.devtools.demo_video` (three trips for the driver flow). Every figure on screen is calculated from those trips; the shape shows the targets met with one branch below target on purpose. **It is simulated, to demonstrate the screens, not a result.**

Appendix A. Feature checklist

Built = works and is tested. **Needs credentials** = code complete, simulated until keys are set.

Feature	Status
Driver app: one big photo button, icons, voice prompts hi / mr / en, installable PWA	Built
Works offline: app opens offline, capture, in-order queue, never twice, other driver actions queued	Built
Photo with GPS and server-time strip; compression under 300 KB; blur / dark / glare / blank checks	Built
Hired-driver private SMS link (one trip, 72 hours)	Needs credentials (Twilio)
AI POD check: LR, stamp, signature, date, quantity, remarks; retake capped at 2	Built
Groq main reader, local Ollama fallback, Gemini, mock; cloud gate	Built
Review queue with labelled boxes on the photo; correction kept beside the AI output	Built
Reopen a rejected POD; cancel an unsent invoice	Built
Billing queue by client and cycle; invoice PDF with POD photos; Tally XML	Built
Push to Tally Prime with duplicate check and ledger creation on yes	Built (Tally must be running)

Payments: manual, part payments, admin undo; trips close when paid	Built
Performance dashboard: no-POD trips, POD-to-invoice by branch, 24-hour rate, lost rate, working capital, trend, digest	Built
Client portal; client POD link; receiver shipment page with delivery code; public tracking link	Built
Daily trip import (CSV or Excel) with LR and e-way bill; preview then confirm	Built
Client request form, quote, yes or no, client login by one-time link	Built
Trip planning board with truck, driver and e-way bill clash checks	Built
LR number generator and LR document (PDF)	Built
Driver GPS with consent, movement states, arrival estimate, live map	Built
Place search and map picker: OpenStreetMap and Photon by default, Google optional	Built
Expenses, advances, Cash-with-you card	Built
Messages between office and drivers; Web Push alerts	Built (push needs keys)
WhatsApp notices in three languages (18 templates)	Built
Driver onboarding by one-time link; staff accounts by admin; PIN change	Built
Setup: trucks with paper expiry, drivers with licences, vendors, clients, branches, message log, filters	Built
Live updates over WebSocket, no reload	Built
Needs-attention list across the shipment	Built
Privacy notice, request register, erasure, 8-year retention job, grievance contact	Built
Audit log; role-checked endpoints; hashed link tokens; rate limits	Built
AI benchmark screen and harness; 28-photo dataset with answer key	Built
English, Hindi, Marathi interface; dark theme; landing page; build-progress page	Built

Where	What
`api/app/pod/decision.py`	The rules from AI reading to verified, retake or review
`api/app/ai/`	AI providers, registry, box locator
`api/app/routers/`	One file per area; every endpoint declares its roles
`api/app/billing/`, `api/app/notify/`	Invoices, PDF, Tally; SMS, WhatsApp, email, push
`web/src/pages/`, `web/src/lib/`	Every screen; offline queue, compression, live updates, i18n
`docs/`	Setup guides per integration, decisions, roadmap, benchmark, DPDP